

Free Form Testing Agent

Explorer Engine Design Document

Version: 1.0

Phase: Phase 2 - Autonomous Exploration Engine

Status: Design Approved

Author: Kalpesh / ChatGPT

Last Updated: June 2026

1. Introduction

The Explorer Engine is the core autonomous component of the Free Form Testing Agent.

Its primary responsibility is to autonomously explore an application's user interface, discover new application states, execute actions, and continuously build a graph representation of the application's behavior.

The Explorer transforms the system from a simple UI automation framework into a true autonomous testing agent.

2. Problem Statement

Traditional UI automation frameworks require human-written scripts:

```
Click Login
Enter Username
Enter Password
Click Submit
Verify Dashboard
```

This approach has several limitations:

- Requires manual test creation.
- Difficult to maintain.
- Limited coverage.
- Cannot discover unknown workflows.
- Cannot autonomously find bugs.
- Cannot adapt to UI changes.

Our goal is different.

Instead of scripting behavior, we want to discover behavior.

3. Explorer Objective

The Explorer must answer the following questions:

What states exist?

```
S0  
S1  
S2  
S3  
...
```

What actions are possible?

```
Click(7)  
Click(+)  
Click(=)  
Input("abc")  
...
```

Which actions lead to new states?

```
S0 --7--> S1  
S1 --+--> S2  
S2 --0--> S3
```

Which actions have already been explored?

```
State:  
  S1  
  
Executed:
```

```
Click(7)
Click(+)
```

```
Not Executed:
Click(8)
Click(-)
```

How can we reach a discovered state?

```
S0
|
7
|
S1
|
+
|
S2
|
0
|
S3
```

4. Explorer Responsibilities

The Explorer Engine is responsible for:

State Discovery

```
Application
  ↓
Capture State
  ↓
Hash State
  ↓
Store State
```

Action Discovery

```
State
  ↓
Find Controls
  ↓
Generate Actions
```

Action Execution

```
Action
  ↓
Executor
  ↓
Application Changes
```

Transition Discovery

```
StateBefore
      +
Action
      +
StateAfter
      ↓
Transition
```

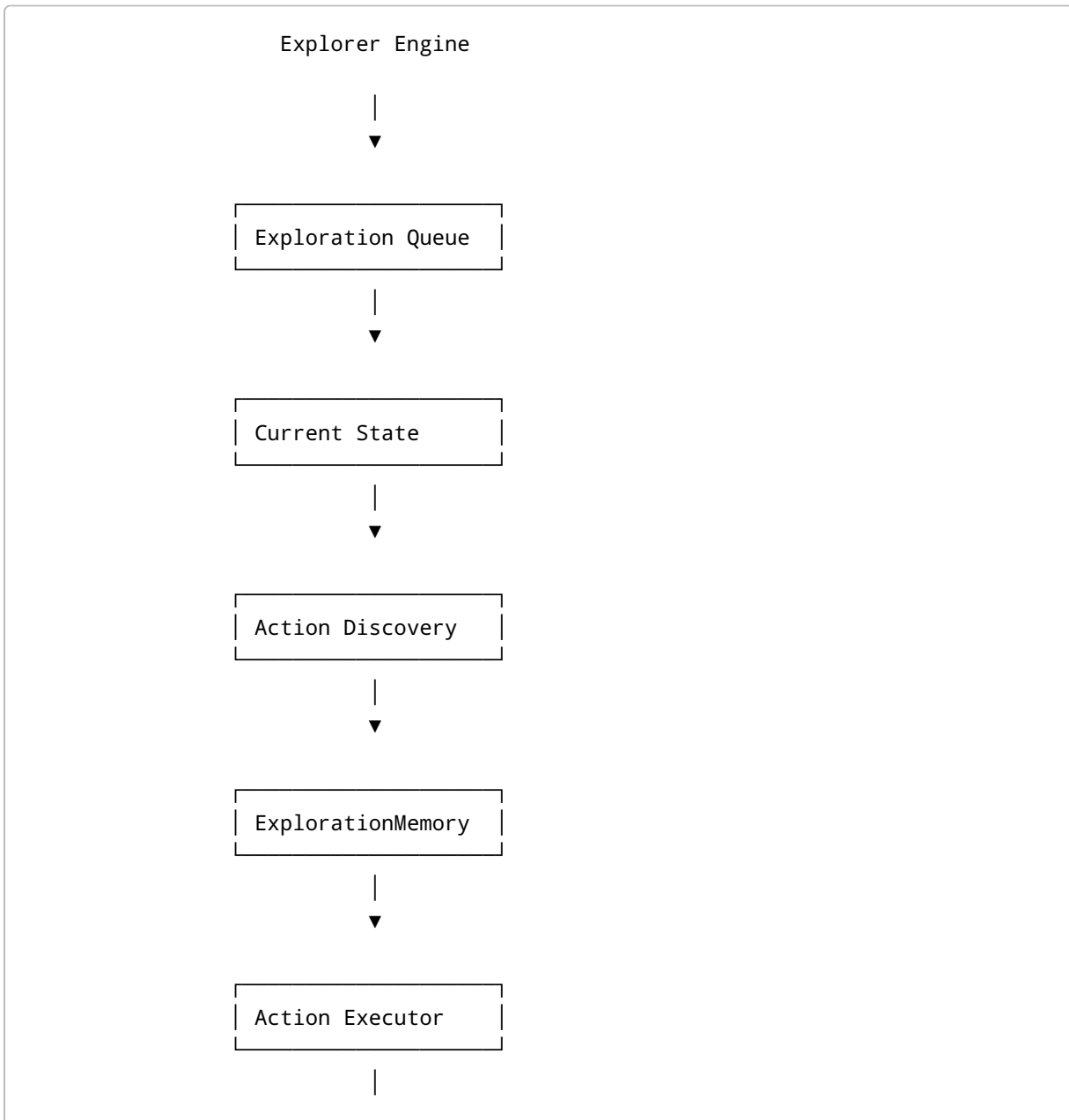
Graph Construction

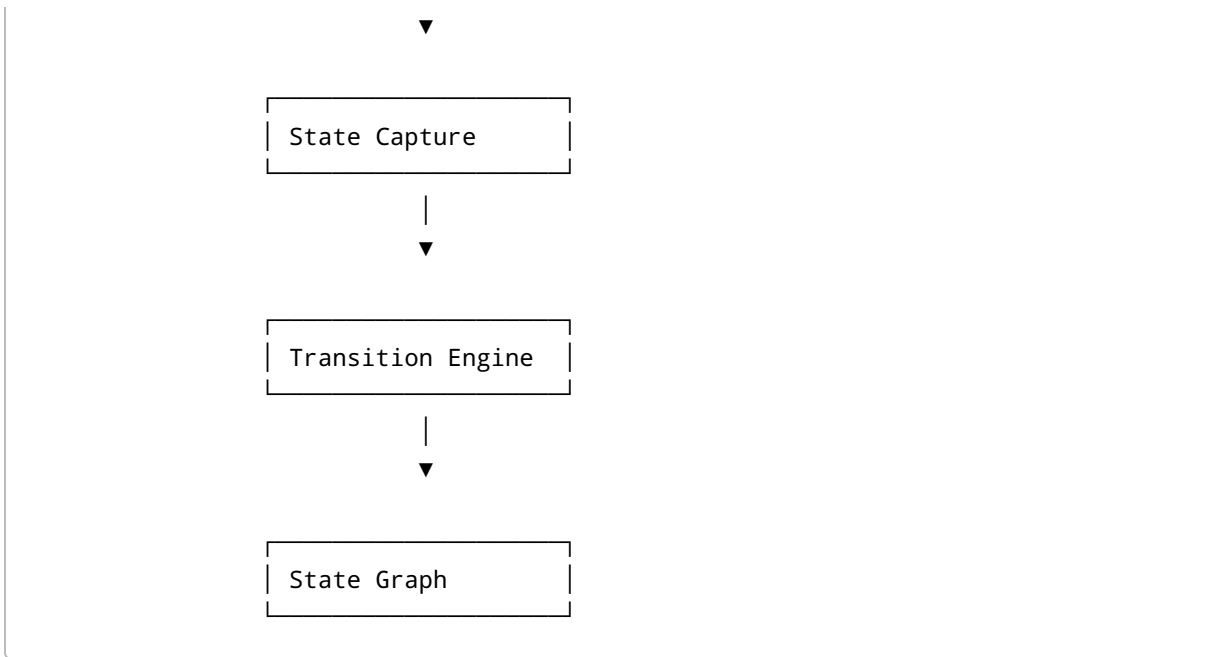
```
State
  ↓
Transition
  ↓
State
```

Coverage Tracking

Discovered States
Discovered Actions
Executed Actions
Unexplored Actions

5. Explorer Architecture





6. Core Exploration Loop

The explorer operates using the following algorithm:

```
while states_to_explore:

    get current state

    ↓

    discover actions

    ↓

    remove already executed actions

    ↓

    choose action

    ↓

    execute action

    ↓
```

capture new state



create transition



update graph



enqueue discovered state

7. Exploration Memory

The explorer never executes the same action twice for the same state.

Example:

State S0

Executed:

num7Button
plusButton
divideButton

Unexplored:

num8Button
num9Button

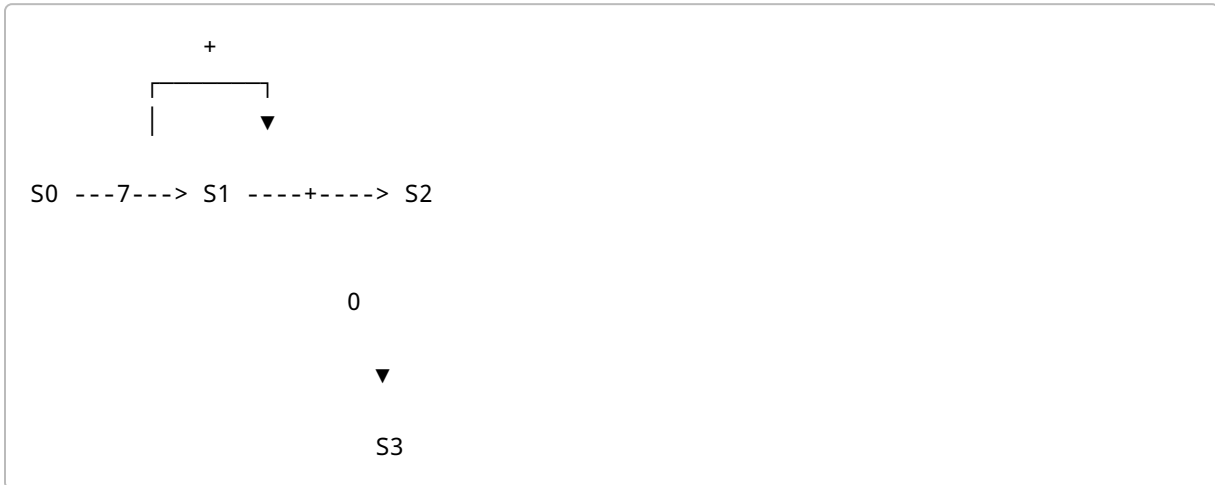
This prevents:

- Infinite loops
 - Duplicate exploration
 - Wasted execution
 - Graph explosion
-

8. State Graph Integration

The explorer continuously builds a graph.

Example:



Graph nodes:

ApplicationState

Graph edges:

Transition

9. Exploration Strategies

The explorer itself does not define exploration behavior.

Instead, multiple strategies can be implemented.

Breadth First Search (BFS)

Queue :

[S0]

Explore:

S0
S1
S2
S3

Advantages:

- Broad coverage.
- Finds shallow bugs.
- Easier visualization.
- Good for GUI crawling.

Depth First Search (DFS)

S0
|
S1
|
S2
|
S3

Advantages:

- Discovers long workflows.
- Reaches hidden states.
- Less memory usage.

Coverage Driven Search

Choose action maximizing:

new_states
+
new_controls
+
new_transitions

Advantages:

- Efficient exploration.
 - Maximum coverage.
-

Reinforcement Learning Search

```
Reward =  
    discovered_state  
    +  
    discovered_bug  
    +  
    discovered_workflow
```

Advantages:

- Self-improving agent.
 - Intelligent exploration.
-

10. BFS Explorer

The first explorer implementation.

Algorithm:

```
queue = [initial_state]  
  
while queue:  
    state = queue.pop()  
  
    actions =  
        get_unexplored_actions()  
  
    for action:  
        execute()  
  
        capture()  
  
        create_transition()
```

```
add_to_graph()
```

```
queue.append()
```

Properties:

Property	Value
Complete	Yes
Optimal	Yes
Deterministic	Yes
Easy Debugging	Yes
Memory Usage	High

11. DFS Explorer

Algorithm:

```
stack = [initial_state]

while stack:

    state = stack.pop()

    execute()

    capture()

    push(new_state)
```

Properties:

Property	Value
Complete	Usually
Optimal	No
Memory Usage	Low
Long Workflow Discovery	Excellent

12. Explorer Components

```
agent/explorer/  
  
    explorer.py  
        Base explorer interface  
  
    bfs_explorer.py  
        Breadth first exploration  
  
    dfs_explorer.py  
        Depth first exploration  
  
    exploration_result.py  
        Exploration statistics  
  
    coverage_engine.py  
        Coverage metrics  
  
    replay_engine.py  
        State replay
```

13. Exploration Statistics

The explorer should continuously collect:

```
States Discovered  
  
Transitions Discovered  
  
Actions Executed  
  
Actions Failed  
  
Coverage %  
  
Execution Time  
  
Average Transition Time  
  
Maximum Graph Depth
```

Cycles Found

Example:

Exploration Summary

States:
125

Transitions:
421

Actions:
489

Coverage:
78%

Depth:
12

Duration:
00:02:35

14. Benefits

The Explorer Engine enables:

Autonomous Test Generation

No human scripting required.

Workflow Discovery

Unknown workflows become known.

Regression Test Generation

Discovered workflows become tests.

Bug Discovery

Unexpected states become bugs.

Replay Support

Any discovered state can be replayed.

Coverage Analysis

Know exactly what has been explored.

LLM Planning

Graph becomes the memory of the AI agent.

15. Future Extensions

Future explorer implementations:

Random Explorer

Heuristic Explorer

Coverage Explorer

Reinforcement Explorer

LLM Planner Explorer

Multi-Agent Explorer

Distributed Explorer

Cloud Explorer

16. Phase 2 Development Plan

Step 1

BFS Explorer

Step 2

Coverage Engine

Step 3

Replay Engine

Step 4

DFS Explorer

Step 5

Autonomous Exploration Runner

Step 6

LLM Planner

17. Final Goal

The final objective of the Explorer Engine is to transform:

Application

into:

Executable Knowledge Graph

where every discovered state, action, transition, workflow, bug, and behavior becomes part of the permanent memory of the autonomous testing agent.